algorithm that, however, will cause the tree to grow beyond what's necessary, or we can implement the full method, more complicated, possibly slower in practice, but with the lowest impact on memory.

Listing 6.7 method Trie::remove (no pruning)

```
function
Trie::remove(s)
      #1
  node ← searchNode(this.root,
s)                                                    #2
  if node == null or node.keyNode == false then
                                #3
    return false

  else
    node.keyNode ←
false                                                          #4
    return true
```

#1 Method `remove` can be implemented naively for the `Trie` class using `search`. Here it takes the key to remove and returns a Boolean conveying the information about whether the key has been found and deleted.

#2 Perform a search on the trie, and get the node for `s` (if present).

#3 If node is `null`, or it's not a key node, it means that the key was not stored in the trie, so we return `false`.

#4 Otherwise, we mark the node as an intermediate node, and return `true`.